



Java Inner Classes

A Comprehensive Guide to Nested Logic

Generated for **codehive**

Introduction

In Java, it is possible to nest classes—defining a class within another class. The purpose of nested classes is to group entities that logically belong together, making your codebase more readable and maintainable.

Why use Inner Classes?

- **Logical Grouping:** If a class is useful to only one other class, it is logical to embed it.
- **Increased Encapsulation:** Inner classes can access private members of the outer class while being hidden from the outside world.
- **Readable Code:** Nesting keeps related code closer together.

1. The Basic Inner Class

To access an inner class, you must first create an object of the outer class, and then create an object of the inner class using the outer object.

Example.java

```
class OuterClass {
    int x = 10;

    class InnerClass {
        int y = 5;
    }
}

public class Main {
    public static void main(String[] args) {
        // 1. Create outer object
        OuterClass myOuter = new OuterClass();

        // 2. Create inner object using the outer instance
        OuterClass.InnerClass myInner = myOuter.new InnerClass();

        System.out.println(myInner.y + myOuter.x);
    }
}
```

Output: 15 (5 + 10)

2. Private Inner Classes

Unlike a "regular" (top-level) class, an inner class can be declared as **private** or **protected**. This is useful when you want to restrict external access completely.

PrivateExample.java

```
class OuterClass {
    int x = 10;

    private class InnerClass {
        int y = 5;
    }
}

public class Main {
    public static void main(String[] args) {
        OuterClass myOuter = new OuterClass();
        // This line will now trigger a compilation error
        OuterClass.InnerClass myInner = myOuter.new InnerClass();
    }
}
```

Compilation Error

If you try to access a private inner class from an outside class, the compiler will stop you:

```
Main.java:13: error: OuterClass.InnerClass has private access in OuterClass
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
            ^
```

3. Static Inner Classes

An inner class can also be **static**. Unlike non-static inner classes, a static inner class does not require an instance of the outer class to be instantiated.

StaticExample.java

```
class OuterClass {
    int x = 10;

    static class InnerClass {
        int y = 5;
    }
}

public class Main {
    public static void main(String[] args) {
        // Note: No 'myOuter' object required here!
        OuterClass.InnerClass myInner = new OuterClass.InnerClass();
        System.out.println(myInner.y);
    }
}
```

Important Note: Just like static methods, a static inner class **cannot** access non-static members (variables or methods) of the outer class directly.

4. Accessing Outer Class Members

One of the primary advantages of (non-static) inner classes is their ability to access attributes and methods of the outer class, even if they are private.

AccessTest.java

```
class OuterClass {
    private int x = 10;

    class InnerClass {
        public int myInnerMethod() {
            // Direct access to outer variable 'x'
            return x;
        }
    }
}

public class Main {
    public static void main(String[] args) {
        OuterClass myOuter = new OuterClass();
        OuterClass.InnerClass myInner = myOuter.new InnerClass();
        System.out.println("Value from Outer: " + myInner.myInnerMethod());
    }
}
```

Comparison Summary

Choosing between Static and Non-Static nested classes depends on your data access needs.

Feature	Inner Class (Non-Static)	Static Nested Class
Outer Instance	Required	Not Required
Access Outer Variables	Can access all (incl. private)	Only static outer members
Instantiation	<code>outerObj.new Inner()</code>	<code>new Outer.Inner()</code>

Exercise & Knowledge Check

Question:

To access a standard (non-static) inner class from a Main method, what steps must you take?

- ☐ A) Create a class, and specify the `friend` keyword.
- ☐ B) Create a class, and specify the `inner` keyword.
- ☐ C) **Create an object of the outer class, and then create an object of the inner class.**

Provided by **codehive** Learning Series